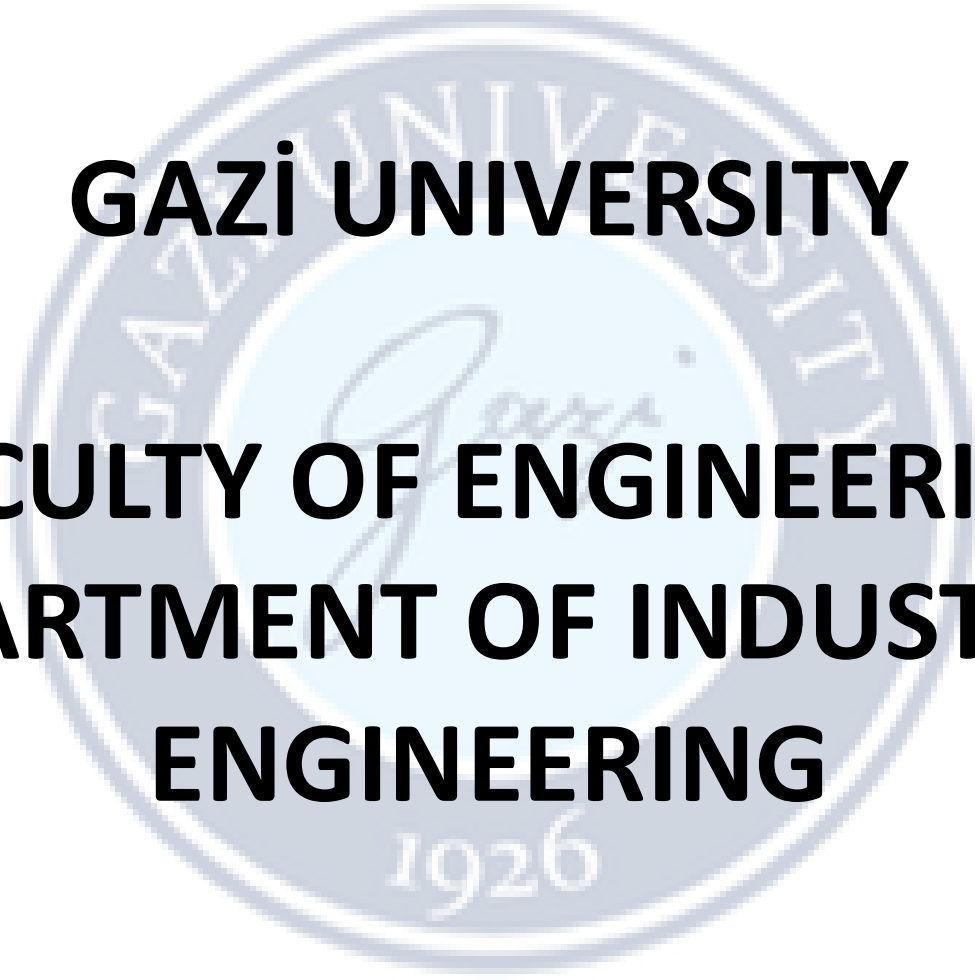


The logo of Gazi University is a circular seal. It features the university's name in Turkish and English around the perimeter, with the founding year '1926' at the bottom. In the center is a stylized signature of 'Gazi'.

GAZİ UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF INDUSTRIAL ENGINEERING

Lecturer : Dr Ercan Ezin

IE326-DATABASE MANAGEMENT SYSTEMS

WEEK 11: Filters-IN, BETWEEN, LIKE, IS NULL

FILTERS



IN, BETWEEN, LIKE, IS NULL

PostgreSQL IN operator

The `IN` operator allows you to check whether a value matches any value in a list of values.

Here's the basic syntax of the `IN` operator:

```
value IN (value1,value2,...)
```



The `IN` operator returns true if the `value` is equal to any value in the list such as `value1` and `value2`.

The list of values can be a list of literal values including numbers and strings.

PostgreSQL IN operator

In addition to literal values, the IN operator also accepts a list of values returned from a query.

Functionally, the `IN` operator is equivalent to combining multiple boolean expressions with the `OR` operators:

```
value = value1 OR value = value2 OR ...
```



PostgreSQL IN operator

- Using IN operator to retrieve information about the film with id 1, 2, and 3:

SAME {
SELECT film_id, title FROM film WHERE film_id IN (1, 2, 3);
SELECT film_id, title FROM film WHERE film_id = 1 OR film_id = 2 OR film_id = 3;

film
* film_id
title
description
release_year
language_id
rental_duration
rental_rate
length
replacement_cost
rating
last_update
special_features
fulltext

The main difference is , compared to OR operator, IN operator is shorter and execution is faster

- Using IN operator to find the film which have the list of exact values:

```
SELECT title FROM film WHERE title IN ('Alien', 'Contact', 'Arrival') ORDER BY title;
```

- Using OR operator to find the film which have certain words in the name:

```
SELECT title FROM film WHERE title LIKE '%Alien%' OR title LIKE '%Contact%';
```

PostgreSQL IN operator

- The following statement uses the IN operator to find payments whose payment dates are in a list of dates: 2007-02-15 and 2007-02-16:

```
SELECT payment_id, amount, payment_date FROM payment WHERE payment_date::date IN ('2007-02-15', '2007-02-16');
```

In this example, the `payment_date` column has the type `timestamp` that consists of both date and time parts.

To match the values in the `payment_date` column with a list of dates, you need to cast them to date values that have the date part only.

To do that you use the `::` cast operator:

```
payment_date::date
```



Output

payment_id	amount	payment_date
17503	7.99	2007-02-15 22:25:46.996577
17504	1.99	2007-02-16 17:23:14.996577
17505	7.99	2007-02-16 22:41:45.996577
17512	4.99	2007-02-16 00:10:50.996577
...		

For example, if the timestamp value is `2007-02-15 22:25:46.996577`, the cast operator will convert it to `2007-02-15`.

PostgreSQL NOT IN operator

To negate the `IN` operator, you use the `NOT IN` operator. Here's the basic syntax of the `NOT IN` operator:

```
value NOT IN (value1, value2, ...)
```



The `NOT IN` operator returns `true` if the `value` is not equal to any value in the list such as `value1` and `value2`; otherwise, the `NOT IN` operator returns `false`.

The `NOT IN` operator is equivalent to a combination of multiple boolean expressions with the AND operators:

```
value <> value1 AND value <> value2 AND ...
```



PostgreSQL NOT IN operator

- The following example uses the NOT IN operator to retrieve films whose id is not 1, 2, or 3:

```
SELECT film_id, title FROM film WHERE film_id NOT IN (1, 2, 3) ORDER BY film_id;
```

Output

film_id		title
-----+-----		
4		Affair Prejudice
5		African Egg
6		Agent Truman
7		Airplane Sierra
8		Airport Pollock

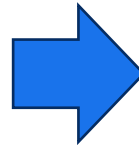
- The following query retrieves the same set of data but uses the not-equal (<>) and AND operators:

```
SELECT film_id, title FROM film  
WHERE film_id <> 1  
AND film_id <> 2  
AND film_id <> 3  
ORDER BY film_id;
```

Same output

PostgreSQL NOT IN operator

```
SELECT
  film_id,
  title
FROM film
WHERE film_id NOT IN (
  SELECT film_id
  FROM maintenance_list
)
ORDER BY film_id;
```



Inner SQL,
Has to be a
list of results,
So only select one column,
film_id
Result: (345, 321,...)

Full SQL

PostgreSQL BETWEEN operator

The `BETWEEN` operator allows you to check if a value falls within a range of values.

The basic syntax of the `BETWEEN` operator is as follows:

```
value BETWEEN low AND high;
```



If the `value` is greater than or equal to the `low` value and less than or equal to the `high` value, the `BETWEEN` operator returns `true`; otherwise, it returns `false`.

You can rewrite the `BETWEEN` operator by using the greater than or equal (`>=`) and less than or equal to (`<=`) operators and the logical AND operator:

```
value >= low AND value <= high
```



PostgreSQL NOT BETWEEN operator

If you want to check if a value is outside a specific range, you can use the `NOT BETWEEN` operator as follows:

```
value NOT BETWEEN low AND high
```



The following expression is equivalent to the expression that uses the `NOT BETWEEN` operators:

```
value < low OR value > high
```



In practice, you often use the `BETWEEN` operator in the WHERE clause of the SELECT, INSERT, UPDATE, and DELETE statements.

PostgreSQL BETWEEN operator

- The following query uses the BETWEEN operator to retrieve payments with payment_id is between 17503 and 17505:

```
SELECT payment_id, amount FROM payment WHERE payment_id BETWEEN 17503 AND 17505 ORDER BY payment_id;
```

payment
* payment_id
customer_id
staff_id
rental_id
amount
payment_date



payment_id	amount
-----+	
17503	7.99
17504	1.99
17505	7.99

- The following example uses the NOT BETWEEN operator to find payments with the payment_id not between 17503 and 17505:

```
SELECT payment_id, amount FROM payment WHERE payment_id NOT  
BETWEEN 17503 AND 17505 ORDER BY payment_id;
```



payment_id	amount
-----+	
17506	2.99
17507	7.99
17508	5.99
17509	5.99
17510	5.99

PostgreSQL BETWEEN operator

If you want to check a value against a date range, you use the literal date in ISO 8601 format, which is

YYYY-MM-DD.

The following example uses the BETWEEN operator to find payments whose payment dates are between 2007-02-15 and 2007-02-20 and amount more than 10:

```
SELECT
  customer_id,
  payment_id,
  amount,
  payment_date
FROM
  payment
WHERE
  payment_date BETWEEN '2007-02-15' AND '2007-02-20'
  AND amount > 10
ORDER BY
  payment_date;
```



customer_id	payment_id	amount	payment_date
33	18640	10.99	2007-02-15 08:14:59.996577
544	18272	10.99	2007-02-15 16:59:12.996577
516	18175	10.99	2007-02-16 13:20:28.996577
572	18367	10.99	2007-02-17 02:33:38.996577
260	19481	10.99	2007-02-17 16:37:30.996577
477	18035	10.99	2007-02-18 07:01:49.996577
221	19336	10.99	2007-02-19 09:18:28.996577

PostgreSQL LIKE operator

Suppose that you want to find customers, but you don't remember their names exactly. However, you can recall that their names begin with something like `Jen`.

How do you locate the exact customers from the database? You can identify customers in the `customer` table by examining the first name column to see if any values begin with `Jen`. However, this process can be time-consuming, especially when the `customer` table has a large number of rows.

Fortunately, you can use the PostgreSQL `LIKE` operator to match the first names of customers with a string using the following query:

```
SELECT
  first_name,
  last_name
FROM
  customer
WHERE
  first_name LIKE 'Jen%';
```



Output:

first_name		last_name
Jennifer		Davis
Jennie		Terry
Jenny		Castro

(3 rows)

PostgreSQL LIKE operator

The `WHERE` clause in the query contains an expression:

```
first_name LIKE 'Jen%'
```



The expression consists of the `first_name`, the `LIKE` operator and a literal string that contains a percent sign (`%`). The string `'Jen%'` is called a pattern.

The query returns rows whose values in the `first_name` column begin with `Jen` and are followed by any sequence of characters. This technique is called pattern matching.

You construct a pattern by combining literal values with wildcard characters and using the `LIKE` or `NOT LIKE` operator to find the matches.

PostgreSQL **LIKE** operator

PostgreSQL offers two wildcards:

- Percent sign (%) matches any sequence of zero or more characters.
- Underscore sign (_) matches any single character.

Here's the basic syntax of the `LIKE` operator:

```
value LIKE pattern
```



The `LIKE` operator returns `true` if the `value` matches the `pattern`. To negate the `LIKE` operator, you use the `NOT` operator as follows:

```
value NOT LIKE pattern
```



The `NOT LIKE` operator returns `true` when the `value` does not match the `pattern`.

If the pattern does not contain any wildcard character, the `LIKE` operator behaves like the equal (=) operator.

PostgreSQL LIKE operator

The following example uses the `LIKE` operator to match any string that starts with the letter `A`:

```
SELECT 'Apple' LIKE 'A%' AS result;
```



Output:

```
result  
-----  
t  
(1 row)
```



The query returns true because the string `'Apple'` starts with the letter `'A'`.

PostgreSQL LIKE operator

- The following example uses the LIKE operator to find customers whose first names contain the string er :

```
SELECT first_name, last_name FROM customer WHERE first_name LIKE '%er%' ORDER BY first_name;
```

customer	
* customer_id	
store_id	
first_name	
last_name	
email	
address_id	
activebool	
create_date	
last_update	
active	

- The following example uses the LIKE operator with a pattern that contains both the percent (%) and underscore (_) wildcards:

```
SELECT
  first_name,
  last_name
FROM
  customer
WHERE
  first_name LIKE '_her%'
ORDER BY
  first_name;
```

Output:

first_name	last_name
Cheryl	Murphy
Sherri	Rhodes
Sherry	Marshall
Theresa	Watson
(4 rows)	



first_name	last_name
Albert	Crouse
Alberto	Henning
Alexander	Fennell
...	...

The pattern `_her%` matches any strings that satisfy the following conditions:

- The first character can be anything.
- The following characters must be `'her'`.
- There can be any number (including zero) of characters after `'her'`.

PostgreSQL **ILIKE** operator

PostgreSQL **ILIKE** operator, which is similar to the **LIKE** operator, but allows for **case-insensitive matching**. For example:

Output:

```
SELECT
  first_name,
  last_name
FROM
  customer
WHERE
  first_name ILIKE 'BAR%';
```



first_name		last_name
Barbara		Jones
Barry		Lovelace

(2 rows)

In this example, the **BAR%** pattern matches any string that begins with **BAR**, **Bar**, **BaR**, and so on. If you use the **LIKE** operator instead, the query will return no row.

SELECT first_name, last_name FROM customer WHERE first_name LIKE 'BAR%'; -> NO row returned

PostgreSQL **LIKE** operator extensions

PostgreSQL also provides some operators that mirror the functionality of `LIKE`, `NOT LIKE`, `ILIKE`, `NOT ILIKE`, as shown in the following table:

Operator	Equivalent	first_name last_name	
~~	LIKE	-----+-----	
~~*	ILIKE	Darlene	Rose
		Darrell	Power
!~~	NOT LIKE	Darren	Windham
		Darryl	Ashcraft
!~~*	NOT ILIKE	Daryl	Larue
		(5 rows)	

For example, the following statement uses the `~~` operator to find a customer whose first names start with the string `Dar`:

```
SELECT first_name, last_name FROM customer WHERE first_name ~~ 'Dar%' ORDER BY first_name;
```



PostgreSQL **LIKE** operator

Sometimes, the data, that you want to match, contains the wildcard characters `%` and `_`. For example:

```
The rents are now 10% higher than last month  
The new film will have _ in the title
```



To instruct the `LIKE` operator to treat the wildcard characters `%` and `_` as regular literal characters, you can use the `ESCAPE` option in the `LIKE` operator:

```
string LIKE pattern ESCAPE escape_character;
```



PostgreSQL **LIKE** operator

The following statement uses the `LIKE` operator with the `ESCAPE` option to treat the `%` followed by the number `10` as a regular character:

```
SELECT * FROM t
WHERE message LIKE '%10$%' ESCAPE '$';
```



message

Data in message column

The rents are now 10% higher than last month
The new film will have _ in the title
(2 rows)

Output:

message



The rents are now 10% higher than last month
(1 row)

In the pattern `%10$%`, the first and last `%` are the wildcard characters whereas the `%` appears after the escape character `$` is a regular character.

Example- Insert the data and answer Q1 & Q2

- `CREATE TABLE employee_directory ( emp_id SERIAL PRIMARY KEY, first_name VARCHAR(50), last_name VARCHAR(50), job_title VARCHAR(100), department_code VARCHAR(10), salary NUMERIC(10, 2) );`
- `INSERT INTO employee_directory (first_name, last_name, job_title, department_code, salary) VALUES ('alice', 'SMITH', 'Lead Software Engineer', 'IT', 95000), ('Bob', 'Smith', 'junior software engineer', 'IT', 55000), ('ALICIA', 'Vance', 'DATABASE ADMINISTRATOR', 'DB', 88000), ('Charlie', 'Smithe', 'Project Manager', 'MGMT', 75000), ('David', 'Miller', 'Senior Engineer', 'IT', 105000), ('Eve', 'Adamson', 'Junior Analyst', 'FIN', 48000);`

Q1: Management needs a list of all employees whose **last name** is exactly 'Smith'. Write a query that retrieves their first_name and last_name using the LIKE operator. Observe how many results you get. Then, write a second query using ILIKE to find all variations (e.g., SMITH, Smith, smith). Concatenate the names into a single column named full_employee_name.

Q2: Retrieve all columns for employees who have the word 'engineer' anywhere in their job_title (regardless of capitalization). Filter the results so that only those with a salary greater than 60,000 are shown and sort the final list by salary in descending order.

PostgreSQL NULL operator

In the database world, NULL means missing information or not applicable. NULL is not a value, therefore, you cannot compare it with other values like numbers or strings.

The comparison of NULL with a value will always result in NULL. Additionally, NULL is not equal to NULL so the following expression returns NULL:

```
SELECT null = null AS result;
```



Output:

```
result
-----
null
(1 row)
```



PostgreSQL IS NULL operator

To check if a value is NULL or not, you cannot use the equal to (`=`) or not equal to (`<>`) operators. Instead, you use the `IS NULL` operator.

Here's the basic syntax of the `IS NULL` operator:

```
value IS NULL
```



The `IS NULL` operator returns true if the `value` is NULL or false otherwise.

To negate the `IS NULL` operator, you use the `IS NOT NULL` operator:

```
value IS NOT NULL
```



The `IS NOT NULL` operator returns true if the value is not NULL or false otherwise.

PostgreSQL **IS NULL** operator examples

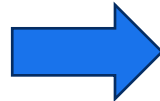
- Please note that the psql program displays NULL as an empty string by default. To change how psql shows NULL in the terminal, you can use the command: \pset null null. It will display NULL as null.

address
* address_id
address
address2
district
city_id
postal_code
phone
last_update

The following example uses the `IS NULL` operator to find the addresses from the `address` table that the `address2` column contains `NULL`:

Output:

```
SELECT
  address,
  address2
FROM
  address
WHERE
  address2 IS NULL;
```



address	address2
-----+	
47 MySakila Drive	null
28 MySQL Boulevard	null
23 Workhaven Lane	null
1411 Lillydale Drive	null

(4 rows)

PostgreSQL IS NOT NULL operator example

The following example uses the `IS NOT NULL` operator to retrieve the address that has the `address2` not NULL:

address
* address_id
address
address2
district
city_id
postal_code
phone
last_update

Output:

```
SELECT
  address,
  address2
FROM
  address
WHERE
  address2 IS NOT NULL;
```



address	address2
1913 Hanoi Way	
1121 Loja Avenue	
692 Joliet Street	
1566 Inegl Manor	

- Notice that the address2 is empty, not NULL. This is a good example of bad practice when it comes to storing empty strings and NULL in the same column.
- To fix it, you can use the UPDATE statement to change the empty strings to NULL in the address2 column. WE will see it later.

DATA FOR QUESTIONS

Create following table and insert the data to answer questions in the following slide:

■ --WEEK 9 & 10: Structural Setup

```
CREATE TABLE logistics_hubs ( hub_id SERIAL PRIMARY KEY, hub_name VARCHAR(100) NOT NULL, city VARCHAR(50), region VARCHAR(20), capacity_teu INT, manager_note TEXT, operating_cost NUMERIC(12, 2) );
```

■ -- WEEK 11: Data Insertion

```
INSERT INTO logistics_hubs (hub_name, city, region, capacity_teu, manager_note, operating_cost) VALUES ('North Star Terminal', 'Istanbul', 'Marmara', 5000, 'Primary export gateway.', 125000.00), ('Aegean Gate', 'Izmir', 'Aegean', 3500, NULL, 85000.50), ('Capital Logistics', 'Ankara', 'Central', 2000, 'High-security inland hub.', 92000.00), ('South Port A1', 'Antalya', 'Mediterranean', 1500, 'Seasonal congestion expected.', 45000.00), ('South Port B2', 'Mersin', 'Mediterranean', 4200, '', 110000.00), ('Eastern Link', 'Erzurum', 'East', 800, 'Cold weather protocols active.', 32000.00);
```

QUESTIONS

1. Retrieve the `hub_name` and `capacity_teu` for all hubs located in the 'Marmara', 'Aegean', or 'Central' regions. Sort the results by `capacity_teu` in descending order.
2. Find the `hub_name` and `city` of any hub whose name contains the word 'Port' (case-insensitive) and has an `operating_cost` between 40,000 and 120,000.
3. List all columns for hubs where the `manager_note` is either missing (NULL) or was recorded as an empty string.
4. Retrieve the top 2 hubs with the highest `capacity_teu` that are located in the 'Mediterranean' region. Use the SQL-standard `FETCH` clause instead of `LIMIT`.

THE END



CHECK OUT COURSE WEBSITE: [HTTPS://GAZI-END-MUH.GITHUB.IO](https://GAZI-END-MUH.GITHUB.IO)

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM